

Draft. Public for testnet testing.

GERO

Geiger Entropy Oracle (GERO)

A randomness oracle on X1 whose node seeds draw on radioactive decay measured by a Geiger counter, combined on-chain with commit-reveal, a future slot hash and a verifiable delay function, so that a consumer's result cannot be computed by anyone before the inputs that fix it exist.

Mainnet: v8.1, live since September 27, 2026

Testnet: v9.1c, live since October 5, 2026 (UTC)

Operator: Echo Hound Labs (single operator, single node today)

Version date: October 5, 2026 (v0.1)

Status: draft. It describes what is deployed and what is planned, and says which is which. Parameters and plans may change; any change will be published.

Important Notice

This document describes the GERO randomness oracle as it is deployed on X1 mainnet and X1 testnet at the version date, and the changes planned for it. It is not an offer of any asset, investment advice or a promise of service levels. GERO is an infrastructure service; using it does not give anyone a share in Echo Hound Labs, GERO or its fees.

No third-party audit and no legal review have been performed. The security review of GERO consists of adversarial audit passes run with Claude Code in fresh sessions (the 2026-10-02 pass with Claude Code (Opus 5.5)), with proof-of-concept tests, recorded in the repository (Section 11). No independent audit firm has reviewed the code, and no lawyer has reviewed this document. None of the passes was performed by an independent firm; the 2026-10-02 report notes that the same model family wrote much of the code under review.

The source code is not public. The repository is private. Build hashes are recorded and the deployed programs have been checked against them by dumping the on-chain bytes (Section 11.3), but outside parties cannot rebuild the programs from source today.

GERO runs with a single operator and a single node today. The protections that need several independent nodes, or an economic penalty that does not flow back to the operator, are not in place yet. Section 9 lists what the operator can and cannot do in each version. Applications should not use GERO for draws whose value exceeds what they are willing to entrust to that operator.

1. Summary

Blockchains are deterministic: every validator must reproduce the same computation, so a smart contract cannot produce an unpredictable number by itself. Randomness has to come from outside, and the question is who can see or steer it before it is used.

GERO answers that with several layers. Its node draws entropy from a GMC-500 Plus Geiger counter measuring background radioactive decay. Each node commits to a hidden seed on-chain before any request that will use it exists, and reveals it later. The seeds are mixed with the hash of a slot that has not been produced when the request is made, and passed through a verifiable delay function (VDF). A consumer program asks for randomness by cross-program invocation; the result is written into a 138-byte request account that the consumer reads back.

Two versions are live:

- **Mainnet runs v8.1** (since September 27, 2026). Each request is bound to one future pool contribution whose seed nobody can compute at request time. Results cannot be predicted before the request's bound slots are produced, but **v8.1 does not claim that results cannot be biased**: the leader of the last bound slot holds every other input when producing it (Section 5.4).
- **Testnet runs v9.1c** (since October 5, 2026). Requests bind to an 8-slot *line*; node seeds stay hidden until after the line's bound slot; the line's input then goes through an RSA-2048 Wesolowski VDF of 10,000,000 sequential squarings whose proof is **verified on-chain**. v9.1c adds the *line record* (LineLog) that the ENTROPY token's minter reads, the N-1 fix (no refund while a request can still be served), and an adjustable request fee paid into a program-held fee pool (Section 6).

The mainnet cutover to v9.1c is planned after the testnet soak and a published gate (Section 13). The honest limits are stated throughout: one operator and one node today, slashing that returns to the operator and therefore does not yet deter, a trusted setup for the RSA modulus, and upgrade keys that are powerful enough to change everything.

Live figures (requests served, fulfilment latency, lines, Geiger count rate) are not quoted in this paper. **See the live GERO dashboard: <https://geiger-dashboard.vercel.app>**

2. Status at a Glance

Item	Mainnet	Testnet
Program	BxUNg2yo5371BQMZPkfcxdCptFRDHkhvEXNM1QNPBRYU	2dQf9uaCzXewrDNLttmtzQmc3SmqfAHz3qahKQjtGQyY
Version	v8.1, upgrade in slot 81,965,230 (September 27, 2026)	v9.1c with the line record, upgrade in slot 193,051,872 (deployed late October 4 US Eastern time, October 5 UTC)
Build	ebb63a0f... (Appendix C)	b67e0f36..., on-chain bytes checked identical (Appendix C)
VDF	chiavdf class-group Wesolowski, 512-bit discriminant, 5,000,000 iterations, computed by the node before commit; not verified on-chain	RSA-2048 Wesolowski, 10,000,000 squarings, computed after the line's last public input; verified on-chain by a separate verifier program
Request fee	none (rent only)	adjustable, 0.05 XNT today, capped at 0.1 XNT in the program, into a program-held fee pool
Refund rule	requester may cancel after 138 slots	refund only when GERO can show the request cannot be served (N-1 fix)
Line record (LineLog)	not deployed	live; read by the ENTROPY minter on testnet
Nodes	one approved node (Genesis)	one node

Item	Mainnet	Testnet
Consumers	RISE Phoenix, Cappy Warriors	Forge Launch (X1 Forge's NFT launchpad)
Upgrade authority	dedicated key held offline, not loaded by any always-on process (since October 4, 2026)	the operator's hot key

Planned: the mainnet cutover to the v9.1c mainnet build (without the line record) after the testnet soak and the cutover gate; the line record on mainnet as part of the ENTROPY launch; more independent nodes; the upgrade authorities moved to a hardware wallet as the last step before real-value use (Section 13).

3. The Problem and Prior Approaches

Commit-reveal. Participants publish hashes of secret values, then reveal them; the combination is the output. The classic weakness is the last revealer, who sees everyone else's values and may decide not to reveal if the outcome is unfavourable [1, 2].

Verifiable random functions. A key holder computes a pseudorandom value with a proof of correctness [3]. Output is verifiable, but the key holder can compute it first and must be trusted not to withhold or to collude [4].

Validator-based beacons. RANDAO-style schemes mix validator contributions; the last contributors can bias the outcome by withholding, at a cost [6, 10].

Verifiable delay functions. A VDF takes a fixed number of sequential steps to evaluate but is cheap to verify [5, 7]. Placed after the last public input, it means nobody can learn the output before a known minimum time has passed, which removes the advantage of whoever produced that input.

Physical sources. Hardware generators based on radioactive decay, photon arrival or thermal noise have long been used off-chain [8, 9]. Their weakness on a blockchain is that nobody else can check where a number came from.

GERO combines a physical source with commit-reveal, future slot-hash binding and a VDF. Physical entropy alone proves nothing to an outside observer, so the on-chain construction is what has to carry the security argument; Sections 5 and 6 describe what it achieves in each version and Section 9 what it does not.

4. Entropy Source

GERO's node uses a GMC-500 Plus Geiger-Müller counter connected over USB serial at 115,200 baud. It detects ionizing radiation from the environment: cosmic-ray secondaries, radon decay products and trace radioactive material. The timing of individual decay events cannot be predicted, even in principle; the daemon uses the timing of detected events as raw entropy, requires at least 5 counts per minute to operate, and checks a device fingerprint at start-up.

How that entropy enters the protocol differs by version:

- **v8.1 (mainnet).** The daemon hashes recent decay data into a seed, runs it through a chiavdf class-group VDF (5,000,000 iterations, about 15 seconds), commits to the output on-chain, and reveals it later. The revealed output, combined with a future slot hash, becomes a pool contribution.
- **v9.1c (testnet).** For every line a node commits to seed = SHA256("GERO_V9_NODE_SEED" || node secret || line || Geiger accumulator || 32 bytes of operating-system randomness) and reveals it after the line's bound slot. If the Geiger feed stops, the daemon raises an alert; the seed is then still unpredictable (node secret and OS randomness), but no longer physically sourced.

What cannot be verified. Nobody outside the node can confirm on-chain that a seed came from a Geiger counter. The on-chain guarantees in Sections 5 and 6 rest on the commitments, the slot hashes and the VDF, not on the physics. The physical source is defence in depth against a weak software random generator; it is not a proof.

5. GERO v8.1 (Mainnet Today)

5.1 Contributions

The node commits to a hash of its VDF output and a nonce, then reveals both. The reveal must land strictly before a *binding slot* about 100 slots later. Once that slot is produced, anyone may finalize the contribution: $\text{pool_seed} = \text{SHA256}(\text{vdf_output} \parallel \text{SlotHash}[\text{binding}] \parallel \text{sequence} \parallel \text{"GEIGER_V6_FINALIZE"})$, written into a 32-entry ring together with a global contribution id. In v8.1 finalization has no approval gate (any signer may send it), a sweeper finalizes contributions the node's own pipeline missed, and an unfinalized contribution cannot be closed while its slot hash is still available.

5.2 Requests

A consumer calls `request_randomness(user_seed)`. The program binds the request to one contribution whose seed does not exist yet: the latest revealed contribution if its binding slot is still in the future, otherwise the next one to be revealed. Fulfilment is refused until that contribution is finalized, and then writes

```
result = SHA256("GEIGER_POOL_V2" || user_seed || request_key || contribution_id || pool_seed || bound_slot || SlotHash[bound_slot])
```

where `bound_slot` is the first produced slot at least 10 slots after the request. If the request is not fulfilled within 138 slots, the requester may cancel it and recover the account rent.

5.3 What v8.1 achieves

Nobody can compute a v8.1 result before both of its bound slots are produced: the node's VDF output was fixed by its commitment before either slot hash existed, and the requester's seed was fixed at request time.

5.4 What v8.1 does not achieve

- **The last leader can bias.** The moment the later of the two bound slots is produced, every other input is public. The validator producing that slot can evaluate the result for each block it could produce and pick one: a few bits, which decides a binary draw. The operator (by commit timing) or a requester (by request timing) can influence whose slot that is, so an operator colluding with one validator, or a validator acting as its own requester, can bias results. **No claim that v8.1 results cannot be biased is made.** v9 was designed to close this.
- **Refund as a re-roll.** Where the operator's own daemon is the only finalizer, not finalizing a contribution makes the bound requests time out and refund. Permissionless finalization and the sweeper reduce this to a liveness question, but consumers should treat a refunded draw as void rather than retrying it against the same stake.
- **No economic penalty.** The v8.1 slash debits the operator's own wallet and can only be executed by the operator; it is not a deterrent.

6. GERO v9.1c (Testnet Today)

6.1 Lines

Time is divided into **lines** of 8 slots. A request made at slot R binds to line $L = \lceil (R + 10) / 8 \rceil$; the line's **bound slot** B is the first produced slot at or after $L \cdot 8$. The request records L and the set of nodes that had committed a seed for L (the *mask*); if no node has, the request is refused at once (`NoEntropyAvailable`) rather than left to fail later. The request account stays 138 bytes, with the same offsets and the same five-account CPI as before, so consumer programs did not have to change their request call.

6.2 Commit, reveal, input

- **Commit.** Each node commits $\text{SHA256}(\text{seed} \parallel \text{nonce})$ for each line in order, and the program refuses a commit once a request for that line could already be made. The mask of every line is therefore fixed before any request of that line exists.

- **Reveal.** The node reveals the seed after B. A reveal counts only if it lands in the window $B < \text{slot} \leq B + 16$. Because no seed is public while B is being produced, the leader of B has nothing to evaluate, whatever its hardware.
- **Input.** Once every masked node has revealed, or the window has closed, anyone may open the line's batch: $x = \text{SHA256}(\text{"GEIGER_V9_INPUT"} \parallel L \parallel B \parallel \text{SlotHash}[B] \parallel (\text{operator} \parallel \text{seed}))$ for each node that revealed on time). A line with no on-time reveal is **dead**; its requests roll over once to a later line chosen only after the dead line is known, and refund only if that line fails too.

6.3 The VDF and on-chain verification

The line's output is $y = g^{(2^T)} \bmod N$ with g derived from x , N the RSA-2048 factoring-challenge modulus and $T = 10,000,000$ sequential squarings, about 10 seconds on the operator's desktop. The VDF sits **after** the last public input, so nobody learns y before doing that sequential work.

The Wesolowski proof is checked **on-chain** by a separate verifier program (HAzgZxKeVPoeCvF76msygeYr5Db3AHNKVYB54ppNMD26) in a set of transactions: one to open an attempt, one to submit y and the proof, ten verification chunks that can be sent in any order, and one to close. X1 has no big-integer modular-exponentiation syscall today, so the 2048-bit arithmetic is done in program code, split to stay under the per-transaction compute limit. The challenge prime is derived from the prover's own key, so a proof copied into another prover's attempt never verifies. Anyone may prove a line; GERO accepts any verified attempt for the line's x .

Each request then gets

```
result = SHA256("GEIGER_P00L_V3" || SHA256("GERO_V9_Y" || y) || request_key || user_seed),
```

written by a fulfil transaction that anyone may send. One VDF serves every request on a line, so the work is bounded by the grid (one line per 8 slots), not by demand.

Latency. The design estimate is about 18 seconds at the median and 25 seconds at the 95th percentile on mainnet slot times (bound slot, reveal window, VDF, verification, fulfil). Measured values: see the live dashboard (<https://geiger-dashboard.vercel.app>).

6.4 The N-1 fix: no refund while a request can be served

In v9 a request's result is fixed once its line's batch is open. Under the older rule, a requester could cancel on time alone after 138 slots, so anyone able to compute the result could refuse a bad draw by waiting, and cheap spam could make many requests expire. v9.1c changes three things:

- `cancel_request` refunds **only** when GERO can show the request cannot be served: its line was never opened in time, the line and its roll target are dead or never opened, or no fulfil path accepts it. Otherwise it fails with `RequestServable`. Anyone may send the cancel; the rent always goes back to the recorded requester.
- Fulfilment has **no deadline** while the line's batch exists (about one day, 250,000 slots); a late fulfil writes the same result an early one would have.
- Every request pays a **non-refundable request fee** (Section 6.5).

On testnet, the rollout record shows a cancel against a live line refused with `RequestServable` and the request then fulfilled at 204 slots.

6.5 Request fee

The fee is set by the oracle authority with `set_request_fee`, between 0 and 0.1 XNT; the cap is enforced by the program. It applies from the next request and is read at request time, so a change never affects a request already made. It is **0.05 XNT today**. The fee is paid into the program's state account and moved by an authority-only sweep into a program-held fee pool (the `["request_fees"]` account). No instruction pays anything out of the pool today; how fees are distributed is a later version's decision and will be published before it ships.

An optional per-request prover bounty (0 by default, at most 0.1 XNT) can be escrowed on the request and paid to whoever proved the line.

6.6 What v9.1c is designed to achieve

The design goal is that no single party, including the leader of the bound slot, can influence a result: the leader sees no seed while producing B; nodes are committed before requests exist; the requester's inputs are fixed at request time; the VDF stops anyone from learning y early; and anyone can finish a line, so no single prover can sit on it. Section 9 lists the residual levers that remain, notably a single node's option to withhold its reveal, and the trust placed in the RSA setup and in the upgrade keys. **This paper does not claim that v9.1c results cannot be biased.**

7. The Line Record (LineLog)

On testnet, GERO writes an on-chain **line record**: a ring of 32,768 entries (about 26.8 hours of lines at the measured mainnet slot time), one per line, stating which node slots revealed on time, whether the line had requests, and whether a proof was verified and by which prover (taken from the verified attempt, never from whoever sent the transaction). Its header lists each node's operator, the line from which it counts, and its payout address. No field of a line's entry is written later than 256 slots after the line starts, so the record of a line is final soon after it closes.

The line record is what the **ENTROPY** token's minter reads. ENTROPY is mined **per line**: every line with at least one on-time node pays its reward to the on-time nodes, to the verified prover if there is one, and to an ecosystem vault. Settlement today is a permissionless crank; moving ENTROPY settlement into the GERO node daemon is planned. On testnet the minter has settled lines against the line record and paid a real claim after the v9.1c upgrade, matching the expected per-line split. For the token itself (supply, schedule, splits, keys, risks) **see the ENTROPY White Paper v0.1.**

The mainnet cutover build of v9.1c does **not** include the line record; it comes to mainnet with the ENTROPY launch, under its own gate.

8. Nodes, Stake and Slashing

From v9, a node must be approved by the oracle authority, placed in the node table (at most 8 slots), and keep an escrowed stake in a program-owned account. The defaults are a **stake of 10 XNT** and a **slash of 1 XNT per withheld line**, within program-enforced ranges of 1-100 XNT and 0.1-10 XNT. A slash change applies only to lines that are not yet committable, so a price cannot be changed retroactively.

A node is slashable when it was in a line's mask and did not reveal on time. The evidence is on-chain (the line's batch), and anyone may report it within 512 slots of the line; the reporter receives 10% and the treasury 90%. A node that goes offline simply stops committing and drops out of later masks; it is not slashed for lines it never committed to.

Honest limit: slashing is not a deterrent yet. The treasury is the oracle authority, which is the operator, and the only node is the operator's own. Slashed XNT therefore returns to the party being slashed. Routing slashes to an escrow that nobody can withdraw from is planned before ENTROPY is initialized on mainnet (see the ENTROPY White Paper); a real deterrent also needs nodes run by other people.

9. Security Model and Honest Limits

9.1 Who can do what

Party	v8.1 (mainnet)	v9.1c (testnet)
Leader of the last bound slot	Holds every other input; can choose among the blocks it could produce. Can bias	Holds no seed while producing B; nothing to evaluate, as long as at least one bound node is honest

Party	v8.1 (mainnet)	v9.1c (testnet)
Operator / node	Cannot predict before the bound slots; with a colluding validator can align timing and bias; can make requests time out if it is the only finalizer	Can withhold its own reveal: with one node that makes the line dead (requests roll over, then refund); with several nodes a withholder chooses between two results (one bit), which needs FPGA-class hardware to evaluate the VDF inside the reveal window. Priced by the slash, which today returns to the operator (Section 8)
Requester	Seed fixed at request; can time its request toward a friendly leader	Seed and key fixed at request; nothing to steer. Refunds only when GERO shows a request unservable (N-1 fix)
Prover	—	Learns y about 10 seconds before others; can withhold, but anyone else can prove the line; a copied proof never verifies
Holder of the RSA-2048 factorisation	—	Can choose every result: with the factors, a valid proof can be forged for any output after x is public. Mitigated only by the modulus's provenance (a public challenge since 1991 whose factors are reported destroyed)
GERO upgrade authority	Can replace the program and so change anything	Same
Verifier upgrade authority	—	As powerful as GERO's: a replaced verifier could mark any attempt verified
Oracle authority	Pause, node approval, pool-age bound	Node table, bounded stake and slash parameters, bounty, request fee (0–0.1 XNT), fee sweep, pause (pause never blocks fulfilment of already-bound requests)

9.2 Limits stated plainly

- **Single operator, single node.** Every protection that relies on "at least one honest node" rests on that one node today.
- **No claim that results cannot be biased,** for v8 and v8.1 in particular, and none for v9.1c beyond the design goal in Section 6.6.
- **Self-slash.** Slashes flow back to the operator; there is no economic deterrent yet.
- **Trusted setup.** The RSA modulus is a trust assumption. A modulus generated by a multi-party ceremony, or a group with no trusted setup that X1 can verify, would remove it; neither is scheduled.
- **Upgrade keys.** Whoever holds the GERO or verifier upgrade authority can change every rule.
- **Physical provenance.** That seeds come from a Geiger counter cannot be checked on-chain (Section 4).
- **Liveness.** If the node stops, requests fail at request time (NoEntropyAvailable) or refund; nobody is penalised for an outage.

9.3 Guidance for consumers

Treat a refund as a void draw, never as a reason to re-roll the same draw. Key any refund in your own program on GERO's decision (a request GERO closed), not on elapsed time. Do not settle a draw worth more than you are willing to trust the operator with, and more than the stake behind it once stake becomes a real deterrent. On v9.1c, the line's batch records which nodes revealed on time, so a consumer can apply its own policy to lines with few or no on-time nodes.

10. Keys and Authorities

Key	Network	Holder today	Power
GERO upgrade authority	Mainnet	A dedicated key held offline (D51c2VbYvhsaHS7ybYnaxrhfaPU4zEhtCgDXpT5SwJy4), moved on October 4, 2026; no daemon, script or service loads it	Replace the program

Key	Network	Holder today	Power
GERO upgrade authority	Testnet	The operator's hot key (FB4jp1T...)	Replace the program
Verifier upgrade authority	Testnet / mainnet (planned)	Testnet: the operator's hot key. Mainnet: the same dedicated offline key as GERO, when the verifier is deployed	Replace the verifier (as powerful as GERO's own key)
Oracle authority	Mainnet / testnet	The operator's keys used by the daemon	Section 9.1
Node key	Both	Hot, in the daemon	Commit and reveal for its node

Moving the GERO and verifier upgrade authorities to a hardware wallet is planned as **the last step before real-value use**. Until then, the trust placed in GERO is trust in the operator and in how it keeps these keys.

11. Security Review and Builds

11.1 Audit policy

No third-party audit firm has been used and no legal review has been performed. Security review means adversarial audit passes run with Claude Code in fresh sessions; none was performed by an independent firm. Each pass is listed below with the reviewer its own report names. Each pass consists of attack hypotheses, source review, and proof-of-concept tests on local validators or LiteSVM against hash-checked binaries, with a written report committed to the repository. The passes so far:

Date and reviewer	Scope	Main findings and status
2026-09-25, Claude Code	Pre-v7 mainnet and v7.1 testnet programs, daemon, documentation	Consumer results did not depend on the entropy pool (C-1); slashing unexecutable by third parties (H-1); operator could drop a contribution after seeing its slot hash (H-2); upgrade key used operationally (C-2). C-1 and H-2 fixed in v8; H-1 and C-2 addressed later (Sections 8, 10)
2026-09-26, Claude Code	v8	Last-leader bias (H-3) and bias by not finalizing (H-4). H-4 reduced in v8.1 by permissionless finalization; H-3 stated as a residual in v8.1 and addressed by the v9 design
2026-09-28, Claude Code	v9.1a on testnet	Pre-funded proof accounts allowed censor-and-reroll (C-1); a secondary RPC endpoint could disrupt reveals (H-1); verifier upgrade key bypasses the verifier lock (H-2); slash parameters unbounded (M-1) and others. C-1, H-1 and the program and daemon Mediums fixed in v9.1b; H-2 and the RSA-setup threat (M-10) are disclosed (Sections 9, 10)
2026-10-02 to 10-04, Claude Code (Opus 5.5)	Mainnet v8.1 → v9.1b cutover	Refund on time alone enabled re-rolls and spam-driven expiry (N-1), fixed in v9.1c; slashes return to the operator (disclosed, Section 8); mainnet upgrade key loaded by the daemon (fixed October 4, 2026)

A further review pass on the exact v9.1c bytes is planned before the ENTROPY launch on mainnet.

11.2 Tests

Each version ships with program tests on a local validator or LiteSVM, migration rehearsals on real mainnet and testnet account bytes (including rollback), the test suite of the Forge Launch consumer run against the GERO binary, and a regression record committed with the build.

11.3 Build hashes

Program builds are made from committed source in fresh directories, reproduced byte-identical, and recorded with their SHA-256 and source commit. Deployed programs are checked by dumping the on-chain bytes and comparing hashes. Because the source is private, outside parties can compare a deployed program with the published hash but cannot rebuild it. Full hashes are in Appendix C.

12. Costs for Consumers

Item	Mainnet (v8.1)	Testnet (v9.1c)
Request account rent (138 bytes)	1,851,360 lamports (0.00185136 XNT); returned to the requester only if the request is cancelled. A fulfilled request account stays open	same
Request fee	none	the fee in force, 0.05 XNT today (cap 0.1 XNT), non-refundable
Prover bounty	—	0 by default (at most 0.1 XNT), refunded with the rent if the request is cancelled
Transaction fees	network fees only	network fees only

Fulfilment and proving are paid by whoever sends them, normally the operator's daemon.

13. Roadmap

Completed (as of October 5, 2026).

- v8.1 live on mainnet since September 27, 2026 (slot 81,965,230).
- v9 designed, built and tested; v9.1a and v9.1b audit fixes; the line record built and run end to end with the real ENTROPY minter on a local validator.
- v9.1c (line record, N-1 fix, adjustable request fee, program-held fee pool) live on testnet since October 5, 2026 (slot 193,051,872), with the fee-to-pool path, the N-1 behaviour and an ENTROPY settle and claim verified after the upgrade.
- Mainnet GERO upgrade authority moved off the daemon's machine to a dedicated offline key (October 4, 2026).

Remaining, in order.

1. Testnet soak of v9.1c: at least 3 days (earliest end October 8, 2026), with no request expiring, no refund on a servable line and the fee accruing to the pool.
2. Mainnet cutover gate: a rehearsal on real mainnet bytes including rollback, signed trust preconditions, funds, and a cutover slot agreed with live mainnet consumers.
3. Mainnet cutover from v8.1 to the v9.1c mainnet build (no line record).
4. Further independent nodes; slashes routed to an escrow nobody can withdraw from.
5. ENTROPY launch on mainnet under its own gate: the line record on mainnet, a further review pass on the exact bytes, ENTROPY settlement in the node daemon. See the ENTROPY White Paper.
6. The GERO and verifier upgrade authorities moved to a hardware wallet: the last step before real-value use.

Possible later work, not scheduled. Request-fee distribution (no payout from the fee pool exists today); a modulus without trusted setup or from a multi-party ceremony; adopting a big-integer syscall if X1 activates one, which would shrink on-chain verification to a few transactions; additional physical entropy sources per node.

14. Risks

- **No third-party audit, no legal review.** Defects missed by every review pass may exist.
- **Operator trust.** Single operator, single node, and upgrade keys that can change every rule (Sections 9, 10).
- **Bias residuals.** v8.1 on mainnet is open to last-leader bias today; v9.1c's design goal rests on at least one honest node, on the RSA setup and on the upgrade keys.
- **No deterrent.** Slashing returns to the operator until slashes go to an escrow and other nodes exist.
- **Liveness.** A node or daemon outage makes requests fail or refund; consumers must handle both.
- **Platform risk.** X1 changes to slot timing, compute limits or program formats could affect latency, verification or the line record.
- **Private source.** Outside parties cannot rebuild the programs from source today.

15. References

- [1] N. Szabo, *Formalizing and Securing Relationships on Public Networks*, First Monday 2(9), 1997.
- [2] I. Bentov, R. Kumaresan, A. Miller, *Instantaneous Decentralized Poker*, ASIACRYPT 2017.
- [3] S. Micali, M. Rabin, S. Vadhan, *Verifiable Random Functions*, FOCS 1999.
- [4] Chainlink Labs, *Chainlink VRF*, technical documentation, 2020.
- [5] D. Boneh, J. Bonneau, B. Bünz, B. Fisch, *Verifiable Delay Functions*, CRYPTO 2018.
- [6] Ethereum Foundation, *EIP-4399: Supplant DIFFICULTY opcode with PREVRANDAO*, 2021.
- [7] B. Wesolowski, *Efficient Verifiable Delay Functions*, EUROCRYPT 2019.
- [8] ID Quantique, *Quantis QRNG*, product documentation.
- [9] J. Walker, *HotBits: Genuine Random Numbers, Generated by Radioactive Decay*, Fourmilab.
- [10] V. Buterin, *Reducing the cost of RANDAO manipulation*, Ethereum Research.
- [11] GQ Electronics, *GMC-500 Plus Geiger Counter User Manual*.
- ENTROPY White Paper v0.1, Echo Hound Labs, October 4, 2026.

16. Glossary

- **Line.** An 8-slot unit of time in v9; each line yields one VDF output shared by all its requests. **Bound slot (B):** the first produced slot at or after the line's start. **Dead line:** no node revealed on time.
 - **Mask.** The set of node slots that had committed a seed for a line when a request was made.
 - **Reveal window.** $B < \text{slot} \leq B + 16$; only reveals inside it count.
 - **VDF.** A function that takes a fixed number of sequential steps to compute and is cheap to verify. **Prover:** whoever computes and submits a line's VDF proof.
 - **Line record (LineLog).** GERO's on-chain ring of per-line entries read by the ENTROPY minter.
 - **N-1 fix.** The v9.1c rule that a request refunds only when GERO can show it cannot be served.
 - **Oracle authority / upgrade authority.** The key that administers GERO's parameters / the key that can replace the program.
-

Appendix A. Parameters

```
v8.1 (mainnet)
FULFILL_MIN_DELAY_SLOTS = 10          CANCEL_WINDOW_SLOTS = 138
BINDING_SLOT_DELAY = 100 slots        pool ring = 32 contributions
VDF: chiavdf class group, 512-bit discriminant, 5,000,000 iterations
request account 138 B                  MIN_CPM = 5

v9.1c (testnet)
GRID = 8 slots per line                line = ceil((request_slot + 10) / 8)
B = first produced slot >= line*8
commit window: (L-32)*8 + 128 < now <= (L-1)*8 - 11
on-time reveal: B < slot <= B + 16
batch open by line*8 + 128            close_line after line*8 + 250,000
slash evidence until line*8 + 512
rollover: one hop, first L' with L'*8 >= B + 26, at most 64 slots later
VDF: RSA-2048 challenge modulus, T = 10,000,000 squarings
verifier: open, prove, 10 chunks, close
stake default 10 XNT (1-100)          slash default 1 XNT per line (0.1-10)
slash split: 10% reporter, 90% treasury (the oracle authority)
request fee 0.05 XNT today (0 to 0.1)  bounty default 0 (at most 0.1)
node table 8 slots
line record: 32,768 entries x 48 B; no write after line*8 + 256
```

Appendix B. Addresses

Item	Address
Dashboard	https://geiger-dashboard.vercel.app
GERO program, mainnet	BxUNg2yo5371BQMZPkfcxdCptFRDHkhvEXNM1QNPBRYU
Oracle state, mainnet	BygMTZ1oLBD9tDmsnt9LkNT7BE2PCJBCzurwtMuTqm
Entropy pool, mainnet	GDECYXCXietabJs9Y1baKzD3t4VFBw4eZWpNvYenyi77
Genesis node, mainnet	z4Psp8qVfP4t3jiWHE29rrisTPMC78tu8LmDhRSEL3s
GERO upgrade authority, mainnet	D51c2VbYvhsaHS7ybYnaxrhfaPU4zEhtCgDXpT5SwJy4
Request-fee pool, mainnet (at cutover)	3Zd147MkBFtBNzyRF5qnJcmA8hQS8K2z2j7g2ScJDvHQ
GERO program, testnet	2dQf9uaCzXewrDNLttmtzQmc3SmqfAHz3qahKQjtGQyY
VDF verifier program, testnet	HAzgZxKeVPoeCvF76msygeYr5Db3AHNKVYB54ppNMD26
Request-fee pool, testnet	68hSchFFG4HrjTxwrEEB3EpLqU8CHPdeAM5YXJXDVB1X

Appendix C. Build hashes

Build	SHA-256	Status
GERO v8.1, mainnet, 545,888 B	ebb63a0f7244d788324763ad9385d5821d89e33f14ef4612d3aeb1cb5267692a	Live on mainnet since slot 81,965,230
GERO v9.1c with line record, testnet, 802,296 B	b67e0f36c047493ca09d663d11e9acaf1f8993e7de12272c7af2386deb347367	Live on testnet since slot 193,051,872; on-chain dump identical
VDF verifier v3, 86,520 B	0a45e37f6bf657988c7ec68a64db7ea8ceffb0c00683bb6d0411c5cfcb2bd81f	Live on testnet
GERO v9.1c, mainnet build (no line record), 696,240 B	4471baeb67eb993b621f5ed6c17832ab2c20de29ab50c37c87b28b700cca3ddd	Candidate for the mainnet cutover, not deployed

Status: Draft, public for testnet testing. No third-party audit, no legal review; no guarantee of availability, latency or absence of bias.